

Live streaming demo

Step-by-Step Presenter Guide

We used this guide in two ways: as a **checklist** to confirm that every component works before our presentation, and as a **script** to follow while we run the live demo in front of our audience. Each step explains what to type, what we should see, and why the step is needed.

1. What This Demo Shows

The demo proves that our project can handle **real-time data**, not only data that has already been stored. Sales records are read from a large file in HDFS, sent one by one into a message queue (Kafka), picked up instantly by a Spark program, and summarised live on screen. Separately, a Django web dashboard shows the predictions made by our trained forecasting model.

1.1 The components and their roles

Component	File / Command	What it does
HDFS	<code>start-dfs.cmd</code>	Hadoop Distributed File System. Stores the 1.1 GB <code>sales_data.csv</code> (about 17.5 million rows) in <code>/project/sales_data</code> .
Kafka	<code>kafka-server-start.bat</code>	A message broker. It receives messages from the producer and holds them in the topic <code>sales-stream</code> until a consumer reads them.
Producer	<code>kafka_producer.py</code>	Reads rows from HDFS using Spark and sends them into Kafka at a controlled speed. This is what “starts” the live stream.
Consumer	<code>spark_streaming_consumer.py</code>	A Spark Structured Streaming job that listens to <code>sales-stream</code> and prints live views of the incoming data and running totals.
Dashboard	<code>manage.py runserver</code>	A Django web app showing KPIs, a predictions table and four charts, read from the <code>daily_sales_predictions</code> data.

1.2 How the data flows

```

sales_data.csv (HDFS)
  | read by Spark
  v
kafka_producer.py --(20 rows/sec)--> Kafka topic: sales-stream
  |

```

```

          v
    spark_streaming_consumer.py
    (live raw records + running totals)

train_and_predict.py --> daily_sales_predictions --> Django dashboard

```

NOTE The bottom line of the diagram is a **separate path**. The dashboard does not read from the live stream. It reads the stored predictions produced earlier by `train_and_predict.py`. This is why you do not need to retrain the model before the streaming demo.

1.3 Terminal windows we will need

We will have several windows open at the same time. Planning them in advance prevents confusion on the day.

#	Window	Started in	Must stay open?
1	HDFS (NameNode and DataNode windows open automatically)	Part A1	Yes
2	Kafka server	Part A2	Yes, do not close it
3	Topic check (short-lived)	Part A3	No, can be closed
4	Spark consumer (live output)	Part B	Yes, keep it visible
5	Producer	Part C	Until it prints "Done"
6	Django dashboard server	Part D	Yes, during the demo

Part A. Prerequisites

Do all of Part A **before** the presentation. These are the background services that everything else depends on. If any of them is not running, the later steps will fail.

A1. Start HDFS

HDFS is where our 1.1 GB sales file lives. The producer cannot send anything if it cannot read this file.

Open a **new Command Prompt** and run:

```
%HADOOP_HOME%\sbin\start-dfs.cmd
```

CHECK Two new windows open (**NameNode** and **DataNode**) and stay running. If they close immediately, HDFS did not start correctly.

Now confirm that the data file is really there:

```
hdfs dfs -ls /project/sales_data
```

CHECK The output lists `sales_data.csv` with a size of about **1.1 GB**.

WHY THIS MATTERS This command asks HDFS to list the folder contents. Seeing the file proves both that HDFS is running and that our data was uploaded correctly.

A2. Start Kafka

Kafka is the “post office” of the demo. It receives messages from the producer and delivers them to the consumer. Open another **new Command Prompt** and run:

```
cd C:\kafka
bin\windows\kafka-server-start.bat config\kraft\server.properties
```

CHECK The log ends with `[KafkaRaftServer nodeId=1] Kafka Server started`.

TIP Leave this window open. Closing it stops Kafka. It is normal for this window to keep printing log lines or to look idle.

NOTE This setup uses **KRaft mode** (the `config\kraft` folder), which means Kafka runs without a separate ZooKeeper service.

A3. Confirm the topic exists

A Kafka **topic** is a named channel that messages are sent to. Our producer and consumer both use the topic `sales-stream`. In a **separate** Command Prompt (Kafka must still be running):

```
cd C:\kafka
bin\windows\kafka-topics.bat --list --bootstrap-server localhost:9092
```

CHECK `sales-stream` appears in the list.

Optional: start with a clean, empty topic

If you ran tests earlier, old messages may still be stored in the topic and could clutter the live view. To start fresh, delete the topic and create it again:

```
bin\windows\kafka-topics.bat --delete --topic sales-stream --bootstrap-server localhost:9092
bin\windows\kafka-topics.bat --create --topic sales-stream --bootstrap-server localhost:9092
```

NOTE Run the delete command first and wait for it to finish, then run the create command. Afterwards, run the list command again to confirm the topic is back.

Part B. Start the Consumer

IMPORTANT Always start the consumer **before** the producer. The consumer must already be listening so it can catch the messages as soon as they are sent.

Open a new terminal (the VS Code terminal or a Command Prompt) and run:

```
cd C:\projects\sales-forecast
bigdata-env\Scripts\activate
python spark_streaming_consumer.py
```

The second line activates the Python virtual environment (`bigdata-env`), which contains the libraries our scripts need, such as PySpark. You should see the following messages, in this order:

1. `Starting Spark session...` This can take about 10 to 20 seconds the first time and is faster afterwards.
2. `Listening to topic 'sales-stream'...`
3. A box saying “Two live views will print below”.

4. **Streaming started. Press Ctrl+C to stop.**
5. Then the screen goes **quiet**. This is correct. The consumer is idle and waiting for new messages.

TIP Keep this terminal window visible during our demo. It is the place where the live output appears.

WHY THIS MATTERS The two live views are (1) a table of the raw records as they arrive and (2) running totals grouped by region and category. Together they show that Spark is both receiving and aggregating data in real time.

Part C. Start the Producer

This is the step that **triggers the live demo**. Open a **second, separate** terminal and run:

```
cd C:\projects\sales-forecast
bigdata-env\Scripts\activate
python kafka_producer.py --rate 20 --limit 500
```

C1. Understanding the flags

Flag	Meaning
<code>--rate 20</code>	Send 20 rows per second . This is slow enough for the audience to watch, yet fast enough that the whole run takes only about 25 seconds.
<code>--limit 500</code>	Stream only 500 rows . This is a manageable demo sample, not all 17.5 million rows in the file.

TIP Adjust the speed to match our presentation. Use a higher rate such as `--rate 100` to finish faster, or a lower rate such as `--rate 5` to get more time to narrate while data flows.

C2. What you will see in the producer terminal

```
Starting producer -> topic 'sales-stream' at 20.0 rows/sec
Starting Spark session to read from HDFS...
Dataset loaded from HDFS: 500 rows. Beginning stream...
Sent 100 / 500 rows (actual rate: 19.8 rows/sec)
...
Done. Sent 500 rows in 25.1 seconds (19.9 rows/sec average).
```

The producer first starts its own Spark session to read the data from HDFS, then sends the rows to Kafka and reports progress every 100 rows. The final “Done” line confirms that everything was sent.

C3. What you will see in the consumer terminal

Within a few seconds of the producer starting, the **consumer terminal (Part B)** begins to react:

- A **batch table of raw records** scrolls in, showing 10 rows per batch.
- Followed by a **batch table of running totals** by region and category, which updates as more data streams in.

DEMO HIGHLIGHT Put the producer and consumer windows **side by side** on screen. The producer sending on one side and the consumer reacting on the other is the actual “real-time” moment of the demo.

Part D. Show the Dashboard

The dashboard presents the results of our trained forecasting model in a visual, easy-to-explain way. Open a new terminal and run:

```
cd C:\projects\sales-forecast\sales_dashboard
..\bigdata-env\Scripts\activate
python manage.py runserver
```

Note the `..\` in the second line. The virtual environment sits one folder above `sales_dashboard`, so the path must step up one level. Then open our browser at <http://127.0.0.1:8000/>.

The three tabs

Tab	What it contains
Overview	KPI summary: number of days evaluated, MAPE (Mean Absolute Percentage Error, the average size of the prediction error), and the best and worst predictions.
Predictions	The full table of predictions, split into pages.
Charts	Four charts: a line chart, a day-of-week bar chart, an error bar chart and an accuracy scatter plot.

NOTE The dashboard reads `daily_sales_predictions`, which was produced by `train_and_predict.py`. It is separate from the live streaming demo, so old-looking data here is fine.

Part E. Clean Shutdown (Optional)

After the demo, press **Ctrl+C** in each of these terminals to stop them:

- The consumer (`spark_streaming_consumer.py`)
- The Django server (`manage.py runserver`)

The Kafka and HDFS windows can stay open or be closed. Closing them simply means you must start them again (Part A) the next time you run the demo.

Quick Troubleshooting During the Demo

If something goes wrong in front of an audience, stay calm and use this table. Most problems come from a service that is not running.

Symptom	Likely cause	Fix
Producer error: “Unable to bootstrap from localhost:9092”	Kafka is not running.	Check that the Part A2 window is still open. Restart Kafka if needed.
Producer error while reading from HDFS	HDFS is not running.	Check that the NameNode and DataNode windows from Part A1 are still open.
Consumer prints nothing after the producer finishes	Spark processes data in micro-batches, so output is not instant.	Wait a few seconds. If nothing appears after 15 to 20 seconds, look for errors in the consumer terminal.
“Address already in use” when starting Kafka	A leftover Kafka process from an earlier session is holding the port.	Find it with <code>netstat -ano findstr :9093</code> , stop it with <code>taskkill /PID <number> /F</code> , then start Kafka again.
Dashboard shows old error data	This is expected.	The dashboard reads <code>daily_sales_predictions</code> from <code>train_and_predict.py</code> . You do not need to retrain for the streaming demo.

Suggested Presentation Order

1. **Prepare in advance.** Have Part A and Part B already running before our audience arrives, so there is no dead air while Spark starts up.
2. **Show the dashboard first (Part D).** It is familiar, visual and easy to explain, so it sets the context for the project.
3. **Switch to the live streaming demo (Part C).** This is the technical highlight. Narrate what is happening as the producer sends rows and the consumer reacts.
4. **Wrap up on the dashboard’s Charts tab.** Tie the real-time pipeline back to the trained model’s predictions to close the story.

Pre-demo checklist

- ☐ HDFS windows (NameNode, DataNode) are open and `sales_data.csv` is listed.
- ☐ Kafka shows “Kafka Server started” and its window is still open.
- ☐ Topic `sales-stream` appears in the topic list (optionally recreated empty).
- ☐ Consumer is running and idle, showing “Streaming started”.
- ☐ Dashboard loads at `http://127.0.0.1:8000/` with all three tabs.
- ☐ Producer and consumer windows are arranged side by side.
- ☐ Producer command is typed or ready to paste with the rate and limit you want.